WARDA

Give AI agents money without giving them unlimited authority.

Spending limits enforced by Kaspa consensus, not by the process holding the key.

Live on Kaspa testnet-10 · MIT · wardaprotocol.com

THE PROBLEM

An agent that can buy things needs money.

So you give it a wallet. Now a process that can be prompt-injected, that you patched last Tuesday, that runs three copies of itself in staging, holds a private key with no spending limit on it at all.

THE WHOLE SECURITY MODEL

The wallet balance

Every other guardrail lives inside the thing being guarded.

WHY THE OBVIOUS FIX FAILS

“Just add a spending cap.”

> “refuses further calls until restarted”
>
> — a production agent-payments library, describing its own spending cap

- ✕ a crash
- ✕ a redeploy
- ✕ a second instance
- ✕ reading the key out of env

A limit enforced by the thing being limited is not a limit. It is a preference.

THE MOVE

Put the limit in the coin, not in the code.

Kaspa's Toccata covenants let an output carry its own spending rules. The money itself knows what it may be spent on.

AGENT

holds a key
and decides

→

GRANT

holds the money
and the rules

→

COVENANT

enforces them
at consensus

→

PAYEE

one of the
committed addresses

A stolen key inherits the limits. That is the difference between a policy and a constraint.

WHAT A GRANT COMMITS TO

Five things, fixed at creation.

01	**Total budget**	the most it can ever spend
02	**Max per payment**	the most one decision can do
03	**Max per epoch**	a rate limit the chain keeps
04	**Authorized payees**	a Merkle root, committed at genesis
05	**Expiry and revocation**	the principal can stop it at any time

The payee list is the one people underestimate. It is not checked against — it is committed to. There is no transaction that pays anyone else. Not one the network rejects: none that exists.

PROOF

We published the private key.

A funded grant on Kaspa testnet with its agent's complete private key printed on a public web page. Not a hash of it — the key. Anyone can sign with it. It has been up for weeks.

MOST ANYONE HAS MANAGED

0

KAS taken outside the grant's authority

Someone tried to drain it at exactly the per-payment cap — the correct attack. The covenant let through exactly the cap and not a sompi more.

wardaprotocol.com/attack

PROOF

Agent #001 is running right now.

It reads the Kaspa network every hour and publishes a daily digest, unattended. Every figure is the difference between two counters the network keeps for its own reasons — nothing sampled, nothing extrapolated.

6

refusals it can prove, not claim

5

of those run the payer's own check

1

authorized payee. Every other address is unpayable

None of those refusal sentences was written for the page. They are produced by running the same check the payer runs before it builds a transaction — so marketing copy that cannot drift, because nobody writes it.

wardaprotocol.com/agent-001

SCOPE

What this does not protect you from.

WARDA GUARANTEES

- ✓ Only an authorized key spends
- ✓ Never above the per-payment cap
- ✓ Never above the epoch allowance
- ✓ Never above the lifetime budget
- ✓ Only to payees committed at genesis
- ✓ Delegation only within the parent's limits
- ✓ Revocation by the principal, at any time

WARDA DOES NOT GUARANTEE

- × That the service you paid for arrives
- × That the vendor is honest
- × That you can get a refund
- × That the price was fair
- × That the agent chose well

Warda is not escrow. It bounds what an agent may spend and says nothing about what it receives. kaspa-x402 built an escrow covenant; the two compose.

FOUND BY SHIPPING

Things no schema could have told us.

~0.02 KAS

Kaspa charges storage mass for small outputs. Payments below roughly 0.02 KAS cannot be broadcast at all — sub-cent micropayments are not a thing on this chain, whatever the budget says.

299×

kaspad sends u64 as raw JSON numbers. Testnet's circulating supply is about 299 times what a double holds exactly, so the value was rounded before our code saw it. The fix had to happen inside the parser.

5 of 5

Payments to a third-party x402 vendor, all accepted on chain, all refused service. Finality, replay and payer address ruled out. Our reading: the exact scheme assumes the payer is a wallet, and a bounded payer is not a wallet.

The third one is unresolved and written up in full, txids included. We would rather publish the open question than a clean demo.

WHERE IT STANDS

Shipped, and honest about what has not.

WORKING TODAY

- Grants, spends, delegation, revocation on testnet-10
- Adversarial probe suite — 14 attacks, all refused
- SDK, x402 adapter and MCP server published to npm
- CLI: zero to a bounded agent in one command
- Browser verifier that re-derives addresses from the covenant

NOT THERE YET

- No mainnet deployment or audit
- No external vendor an agent can successfully buy from
- Delegation demonstrated in code, not yet in public
- One authorized payee on the live agent, not twelve

The authority layer is ready. The market for agent services is not — x402 across all chains does roughly $28k a day and about half of that is self-dealing. We would rather be early and say so.

CHECK IT YOURSELF

The dashboard is not the source of truth.

Kaspa is. Everything we have claimed is re-derivable from the chain.

```
npm i -g warda
warda init --rpc wss://<your-node>
warda grant create --budget 10 --max-spend 0.1 --payee <address>
warda buy https://<any-x402-endpoint>
```

wardaprotocol.com	the protocol, the attack, agent #001
github.com/ArtyKOMarkets/warda	MIT. Every claim here has a file behind it.